SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 — Git & Docker Technical Assignment

Course Code:	CSA1007	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	*Build full-stack applications with an emphasis on containerization, version control, and collaborative development.*		
Student Name:	N. Madhusudhan	**Reg. No.:**	192425303
Date:	08/08/2026	**Duration:**	60 minutes

Code of Conduct

I, N. Madhusudhan (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N. Madhusudhan

Problem Statement 26: Smart Smart Grid Cybersecurity Management Platform

1. Problem Analysis

The assigned project is a Smart Grid Cybersecurity Management Platform that continuously monitors grid security, detects cyber threats, predicts attack patterns, tracks network activities, generates regulatory compliance reports, and notifies security teams in real time. The core project objectives are to protect critical grid infrastructure (substations, SCADA systems, and control networks) from cyber intrusions while giving security operations teams accurate, timely, and auditable visibility.

- Functional requirements: real-time grid security monitoring, AI/ML-based threat and anomaly detection, network activity tracking, predictive attack-pattern analytics, compliance reporting, and instant security alerting.
- Expected outcomes: improved grid security, reduced cyber incidents, faster threat response, enhanced infrastructure resilience, and better regulatory compliance.

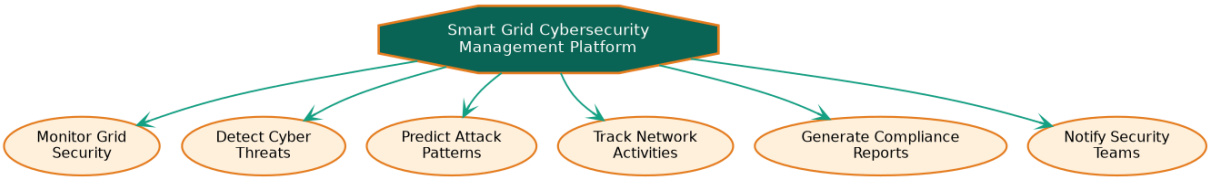

Figure 1: Core system objectives derived from the problem statement.

2. Project Structure Design

The repository is organized as a microservices monorepo, with one folder per security capability plus shared infrastructure and documentation folders, so that each module can be built, tested, and containerized independently.

- Service folders (threat-detection-, network-monitoring-, attack-prediction-, compliance-, notification-, dashboard-service): isolate each security capability's code, tests, and Dockerfile.

- api-gateway/ and frontend/: house the single entry point and the Security Operations Center (SOC) web UI, kept separate from backend security logic.
- infra/: centralizes docker-compose.yml and reverse-proxy configuration so orchestration is not scattered across services.
- .github/workflows/, docs/, .gitignore, README.md: support CI automation, architecture documentation, clean version control, and onboarding.

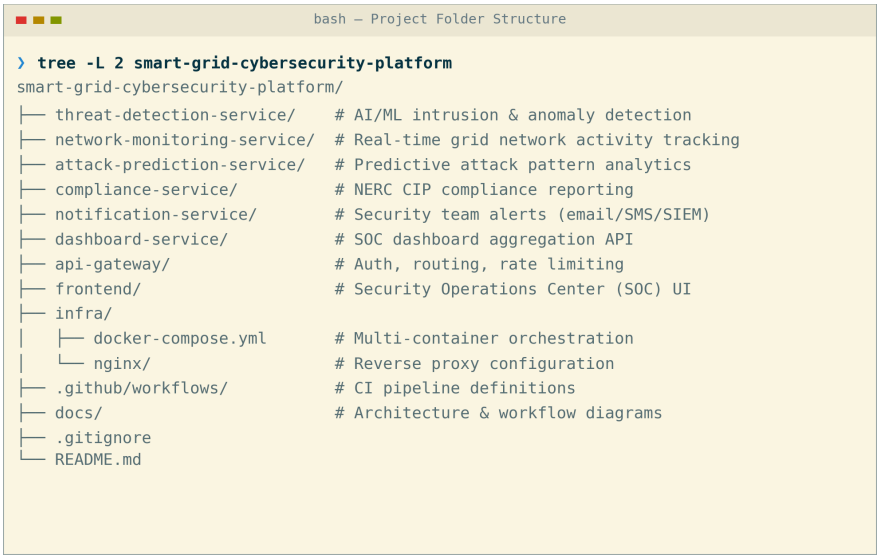

Figure 2: Repository folder structure.

3. Git Repository Initialization

The repository was initialized with git init, followed by creating a README.md and a .gitignore before the first commit, so that history begins from a clean, documented baseline. The repository was then linked to a remote and pushed to establish the shared origin for the security engineering team.

- .git/ — hidden metadata directory Git uses to track history, branches, and configuration; created automatically by git init.
- .gitignore — excludes build artifacts, node_modules, .env secrets, and other machine-specific files from version control.
- README.md — documents project purpose, setup, and structure for new contributors.

Figure 3: Git repository initialization sequence.

4. Git Branching Strategy

A Git-Flow-inspired strategy was adopted: a stable main branch, an integration develop branch, short-lived feature/* branches per security capability, release/* branches for stabilization, and hotfix/* branches for urgent production fixes.

- main only ever holds production-ready, tagged releases, keeping deployment risk low for a security-critical platform.
- develop integrates completed features before a release is cut, giving a stable point for cross-service integration testing.
- feature/* branches (e.g., feature/threat-detection-service) isolate each microservice's work, matching the project's modular architecture and letting multiple security engineering teams work in parallel without blocking each other.
- hotfix/* branches patch main directly and back-merge into develop, so urgent security fixes reach production fast without waiting for the next release.

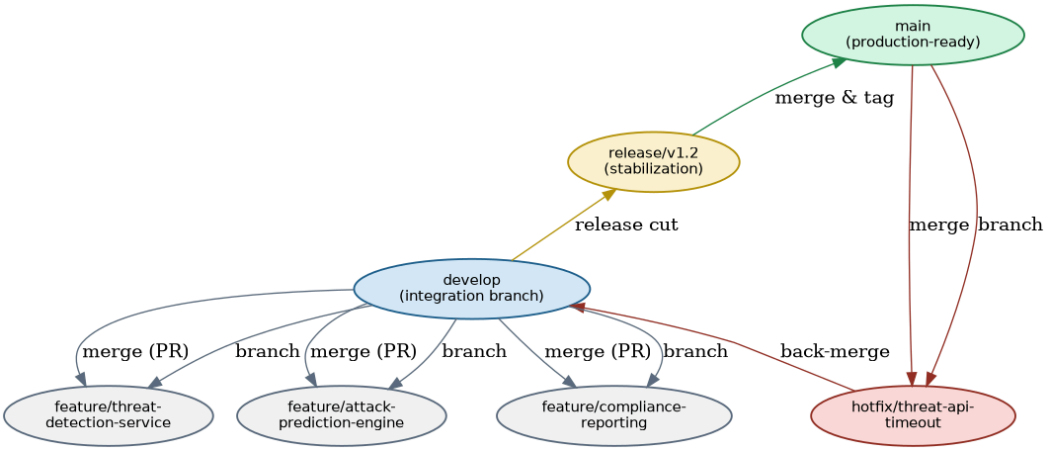

Figure 4: Git branching strategy.

5. Version Control Workflow

The workflow below demonstrates a complete cycle: branching off develop, committing meaningful changes, merging back with conflict resolution, and synchronizing with the remote repository.

- git checkout -b creates an isolated branch so in-progress work never destabilizes develop.
- git commit with a scoped message records an atomic, traceable change to the threat detection service.
- git merge --no-ff preserves branch history; the resulting conflict in app.py was resolved manually and re-committed, demonstrating safe conflict handling.
- git pull --rebase and git push keep the local and remote develop branches synchronized without creating unnecessary merge commits.

```
bash — Version Control Workflow

# Create and switch to a feature branch
› git checkout -b feature/threat-detection-service develop
Switched to a new branch 'feature/threat-detection-service'

› git add threat-detection-service/
› git commit -m "feat(threat): add anomaly detection scoring endpoint"
[feature/threat-detection-service 2c9d4e1] feat(threat): add anomaly detection endpoint

# Merge feature branch back into develop
› git checkout develop
› git merge --no-ff feature/threat-detection-service
Auto-merging threat-detection-service/app.py
CONFLICT (content): Merge conflict in threat-detection-service/app.py
# Conflict resolved manually, then staged and committed
› git add threat-detection-service/app.py
› git commit -m "merge: resolve conflict in threat-detection-service/app.py"
[develop 5e6f7a8] merge: resolve conflict in threat-detection-service/app.py

# Synchronize with the remote repository
› git pull origin develop --rebase
› git push origin develop
develop -> develop  (up to date)
```

Figure 5: End-to-end Git workflow — branch, commit, merge, conflict resolution, sync.

6. Commit History Analysis

The commit graph below shows a linear main history feeding into develop, with a feature branch merged back through an explicit merge commit. Every message follows the Conventional Commits format, <type>(<scope>): <description>, which keeps history searchable and self-documenting.

- Consistent types (feat, fix, chore, merge) let the team filter history by change category during release notes generation.
- Scoped messages (e.g., threat, network, gateway) trace each commit directly to the affected security module, simplifying debugging and audit review.

```
bash — Commit History

› git log --oneline --graph --decorate --all
* 5e6f7a8 (HEAD -> develop) merge: resolve conflict in threat-detection-service/app.py
|\
| * 2c9d4e1 (feature/threat-detection-service) feat(threat): add anomaly endpoint
| * 1b8c3d2 feat(threat): scaffold threat detection microservice
|/
* 0a7b2c1 feat(network): add network activity ingestion pipeline
* 9f6e1d0 feat(gateway): configure API gateway routing
* 8e5d0c9 (main) chore: initial commit with README and .gitignore

# Commit messages follow the Conventional Commits style:
# <type>(<scope>): <short, imperative description>
# e.g. feat, fix, chore, docs, refactor, test — each traceable to a module
```

Figure 6: Commit history (git log --oneline --graph --all).

7. Docker Environment Design

Docker is well suited to this project because the platform is composed of independently deployable, heterogeneous microservices (a compute-heavy AI/ML threat-detection service alongside lightweight transactional services) that must run consistently across developer machines, CI, and production, while remaining tightly access-controlled for a security-critical system.

- Containerization eliminates environment drift between developers and deployment targets.
- Each service scales independently (e.g., extra threat-detection replicas during elevated alert periods) without over-provisioning lighter services.

- Components requiring containerization: the six backend microservices, the API gateway, the SOC frontend, and supporting infrastructure (PostgreSQL, Redis, Kafka).

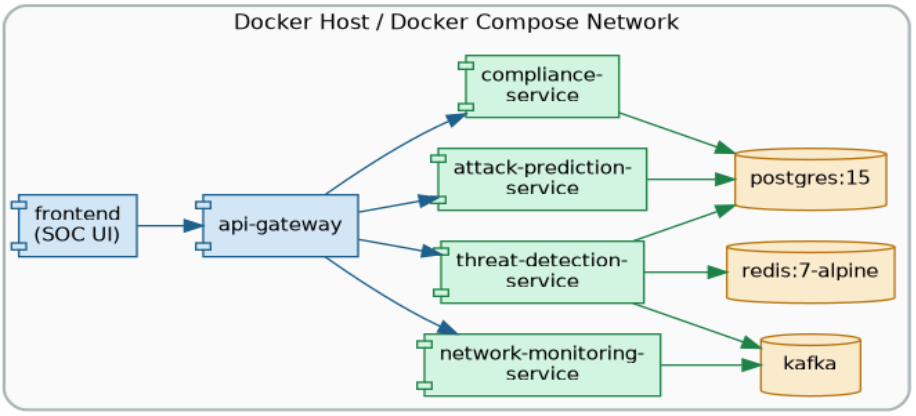

Figure 7: Components requiring containerization and their interactions.

8. Dockerfile Development

The Dockerfile below containerizes the threat detection service using a slim Python base image, installs dependencies before copying source code to maximize layer-cache reuse, runs as a non-root user for security, and defines a health check so the orchestrator can detect failures.

- FROM selects a minimal, version-pinned base image to reduce attack surface and image size.
- WORKDIR / COPY / RUN order is chosen deliberately so dependency installation is cached separately from source code changes.
- USER appuser drops root privileges inside the container, following container security best practice — critical for a cybersecurity platform.
- HEALTHCHECK and CMD define how the container reports readiness and how it starts the application server.

Dockerfile — threat-detection-service

```
# Dockerfile — threat-detection-service
FROM python:3.11-slim AS base

# Set working directory inside the container
WORKDIR /app

# Install dependencies first (leverages Docker layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code
COPY . .

# Create a non-root user for security
RUN useradd -m appuser && chown -R appuser /app
USER appuser

# Document the container's listening port
EXPOSE 5001

# Container health check
HEALTHCHECK --interval=30s CMD curl -f http://localhost:5001/health || exit 1

# Start the threat detection service
CMD ["gunicorn", "-b", "0.0.0.0:5001", "app:server"]
```

Figure 8: Dockerfile for the threat-detection-service.

9. Docker Compose Design

Docker Compose orchestrates all services on a shared bridge network so containers can reach each other by service name, while only the gateway and frontend expose ports to the host.

- Services: gateway, threat-detection, network-monitoring, postgres, redis, and kafka, each built or pulled independently.
- Networking: a single backend bridge network isolates inter-service traffic from the host network, exposed only through the gateway.
- Volumes: a named pgdata volume persists PostgreSQL compliance and audit data across container restarts.
- Environment variables: DB_URL is injected at runtime (with secrets externalized), keeping credentials out of the image.
- Dependencies: depends_on ensures postgres, redis, and kafka start before the services that require them.

docker-compose.yml

```yaml
version: "3.9"
services:
  gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    depends_on: [threat-detection, network-monitoring, postgres]
    networks: [backend]

  threat-detection:
    build: ./threat-detection-service
    environment:
      - DB_URL=postgresql://grid:${DB_PASS}@postgres:5432/grid
    depends_on: [postgres, redis, kafka]
    networks: [backend]

  network-monitoring:
    build: ./network-monitoring-service
    depends_on: [kafka]
    networks: [backend]

  postgres:
    image: postgres:15
    volumes: ["pgdata:/var/lib/postgresql/data"]
    networks: [backend]

  redis:
    image: redis:7-alpine
    networks: [backend]

  kafka:
    image: bitnami/kafka:3.6
    networks: [backend]

volumes:
  pgdata:
networks:
  backend:
    driver: bridge
```

Figure 9: docker-compose.yml configuration.

10. Container Deployment and Testing

Running docker-compose up --build -d builds and starts every container in dependency order. docker ps confirms each container reports a healthy status, and a functional test against the gateway's threat-score endpoint confirms the threat detection service is reachable end-to-end and returning a valid response.

- Deployment evidence: all containers reach 'healthy' status within the expected startup window.
- Testing procedure: a curl request through the API gateway to the threat detection service validates routing, service health, and response correctness in one step.

```
bash — Container Deployment & Testing

# Build and start all services
❯ docker-compose up --build -d
 ✔ Network smart-grid-scm_backend       Created
 ✔ Container smart-grid-postgres        Started
 ✔ Container smart-grid-redis           Started
 ✔ Container smart-grid-kafka           Started
 ✔ Container smart-grid-threat-detect    Started
 ✔ Container smart-grid-network-monitor  Started
 ✔ Container smart-grid-gateway          Started
 ✔ Container smart-grid-frontend         Started

# Verify all containers are healthy
❯ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                     STATUS                   PORTS
smart-grid-gateway        Up 2 minutes (healthy)   0.0.0.0:8080->8080/tcp
smart-grid-threat-detect  Up 2 minutes (healthy)   5001/tcp
smart-grid-network-monitor Up 2 minutes (healthy)  5002/tcp
smart-grid-frontend       Up 2 minutes (healthy)   0.0.0.0:3000->3000/tcp
smart-grid-postgres       Up 2 minutes (healthy)   5432/tcp

# Functional test: request a threat score via the gateway
❯ curl -s http://localhost:8080/api/threat-score/SUBSTATION-014 | jq
{ "assetId": "SUBSTATION-014", "threatScore": 0.12, "status": "normal" }
200 OK — threat-detection service responded successfully
```

Figure 10: Container deployment and functional test evidence.

11. Benefits of Git and Docker

Together, Git and Docker directly address the collaboration and operational challenges of a multi-service, security-critical smart grid platform, summarized below.

Dimension	**Benefit from Git**	**Benefit from Docker**
Collaboration	Parallel feature branches let multiple security teams work simultaneously without blocking each other.	Identical local dev containers eliminate "works on my machine" issues across the team.
Version Management	Full history and tags (v1.0, v1.1) enable safe rollback of any microservice independently.	Image tags version the exact runtime environment alongside the code.
Portability	Repository clones identically on any developer machine or CI runner.	Containers run identically on a laptop, CI runner, or any cloud provider.
Deployment	Protected main branch + PR review gates what reaches production.	docker-compose up / CI pipeline gives fast, repeatable, one-command deployment.
Scalability	Feature branches isolate work so new detection modules can be added without disrupting others.	Each microservice container can be scaled independently (e.g., extra threat-detection replicas).
Maintainability	Commit history and blame trace exactly when and why a change was made.	Dockerfiles document the runtime setup, removing manual server configuration drift.

12. Challenges and Improvements

Implementation surfaced several practical challenges, each with a corresponding improvement identified for future iterations of the Git workflow and Docker deployment.

Challenge Encountered	**Suggested Improvement / Future Enhancement**
Merge conflicts when multiple feature branches touched shared service code.	Adopt smaller, more frequent PRs and enforce code-owner review on shared modules.
Large Docker image sizes slowed down build and deployment time.	Use multi-stage builds and slim/alpine base images to reduce final image size.
Managing environment-specific secrets (DB passwords, API keys).	Introduce a secrets manager (e.g., Docker Secrets / Vault) instead of plain .env files.
Coordinating release timing across multiple security microservices.	Add automated CI/CD pipelines with staged rollouts and semantic-version release tagging.
Limited visibility into container resource usage during scaling.	Integrate container monitoring (Prometheus + Grafana) for CPU/memory-based auto-scaling.